Clasificación de dibujos: Google Quick, Draw!

Objetivo del Trabajo Práctico 02

Evaluar lo visto en clase sobre clasificación, selección de modelos utilizando validación cruzada y cómo estimar la performance que va a tener un modelo al usarlo con datos que no han sido vistos.

Enunciado

En el presente TP trabajaremos con un subconjunto del conjunto de datos de imágenes denominado Quick, Draw! [1]. Este dataset fue construido a partir de los dibujos que realizaron millones de personas jugando al juego online "Quick, Draw!" de Google: en cada partida, el juego le pide al jugador que dibuje un objeto en menos de 20 segundos, mientras una red neuronal intenta adivinar qué está dibujando. Cada imagen del set de datos que vamos a utilizar representa un dibujo hecho a mano de un objeto de entre 10 tipos distintos (por ejemplo, un círculo, una escalera, un pez o una casa). Para este TP nos quedamos únicamente con los dibujos que el juego logró reconocer, y los llevamos a imágenes en escala de grises de 28x28 píxeles.
En el link ubicado a pie de página pueden acceder a una descripción más detallada del dataset original. Ahí pueden ver todas las categorías disponibles.
Para comenzar deben **descargar del campus de la materia** el conjunto de datos. Encontrarán tres archivos en formato .csv. El archivo “clases.csv” tiene el mapeo de cada clase con el nombre del objeto dibujado (en inglés) e “imagenes.csv” tiene la representación en píxeles y la clase de cada imagen. El tercer archivo, “evaluacion.csv”, contiene el conjunto de evaluación de la cátedra: son imágenes nuevas, distintas de las anteriores, de las que sólo tienen la representación en píxeles (junto con un identificador), pero no la clase. Este archivo se utiliza únicamente en el Ejercicio 3.e.

Fecha límite para la entrega: Domingo 01 de noviembre, 23:50hs. Al igual que el TP-01, la entrega de este TP se realizará a través del campus de la materia. El día jueves 05 de noviembre, antes del fin de la clase (16hs), deberán entregar también una versión impresa del informe.

Ejercicios

1. Realizar un análisis exploratorio de los datos. Entre otras cosas, deben analizar la cantidad de datos, cantidad y tipos de atributos, cantidad de clases de la variable de interés (objeto dibujado) y otras características que consideren relevantes. Además,

se espera que con su análisis puedan responder las siguientes preguntas:

a. ¿Cuáles pueden ser atributos relevantes para predecir el objeto al que corresponde el dibujo? ¿Cuáles no? ¿Creen que se pueden descartar atributos?
b. ¿Hay objetos cuyos dibujos son más parecidos entre sí? Por ejemplo, ¿Son más fáciles de diferenciar las imágenes correspondientes a la clase 2 (cuadrado) de las de la clase 9 (casa) que de las de la clase 6 (estrella)?
c. Tomar una de las clases, por ejemplo la clase 3 (pez), ¿Son todas las imágenes muy similares entre sí?
d. Este dataset está compuesto por imágenes, esto plantea una diferencia frente a los datos que utilizamos en las clases (por ejemplo, el dataset de Titanic). ¿Creen que esto complica la exploración de los datos?

Importante: las respuestas correspondientes a los puntos 1.a, 1.b y 1.c deben ser justificadas en base a gráficos de distinto tipo.

Ayuda: Para ayudarles en la representación gráfica les dejamos código para orientarlos.

```
###########
# Plot imagen
img = np.array(X.iloc[12]).reshape((28,28))
plt.imshow(img, cmap='gray')
plt.show()
############
```

2. **(Clasificación binaria)** Dada una imagen se desea responder la siguiente pregunta: ¿corresponde esta imagen a la clase 2 (cuadrado) o a la clase 9 (casa)?
   a. A partir del dataframe original, construir un nuevo *dataframe* que contenga sólo al subconjunto de imágenes correspondientes a las clases 2 y 9. Sobre este subconjunto de datos, analizar cuántas muestras se tienen y determinar si está balanceado con respecto a las dos clases a predecir.
   b. Elegir las métricas de evaluación para problemas de clasificación que van a utilizar para comparar los modelos de este ejercicio. Para ello, tener en cuenta lo analizado en el inciso anterior (por ejemplo, si el conjunto está balanceado) y qué tipos de errores puede cometer el modelo. Explicar por qué decidieron utilizar esas métricas y no otras.
   c. Separar los datos en conjuntos de *train* y *test*.
   d. Ajustar un modelo de KNN sobre los datos de entrenamiento utilizando una cantidad reducida de atributos (por ejemplo, 3). Probar con distintos conjuntos de atributos seleccionados a partir del análisis exploratorio -por ejemplo, varios subconjuntos distintos de 3 atributos si se eligió ese número- y comparar los resultados obtenidos. Repetir el análisis utilizando diferentes cantidades de atributos. Para comparar los resultados de cada modelo usar el conjunto de test generado en el punto anterior y las métricas elegidas en el inciso b.
   e. Comparar modelos de KNN utilizando distintos atributos y distintos valores de k (vecinos). Para el análisis de los resultados, tener en cuenta las métricas de evaluación utilizadas y la cantidad de atributos utilizados.

3. **(Clasificación multiclase)** Dada una imagen se desea responder la siguiente pregunta: ¿A cuál de las 10 clases corresponde la imagen?
    a. Separar el conjunto de datos en desarrollo (*dev*) y control (*held-out*). Para el inciso c, utilizar únicamente el conjunto de datos *dev*. Dejar apartado el conjunto *held-out* en este inciso.
    b. Elegir las métricas de evaluación para problemas de clasificación multiclase que van a utilizar para comparar los modelos de este ejercicio. Explicar por qué decidieron utilizar esas métricas y no otras. Además, pueden realizar una matriz de confusión y evaluar los distintos tipos de errores para las clases.
    c. Realizar un experimento para comparar y seleccionar árboles de decisión con distintos hiperparámetros, incluyendo distintas profundidades (limitarse a profundidades entre 1 y 10). Para esto, utilizar validación cruzada con *k-fold*. ¿Cuál fue el mejor modelo? Documentar cuál es la mejor configuración de hiperparámetros, y qué desempeño tiene según las métricas elegidas en el inciso b.
    d. Entrenar el modelo elegido a partir del inciso previo en todo el conjunto de desarrollo. Utilizarlo para predecir las clases en el conjunto *held-out* y reportar el desempeño.
    e. **(Estimación del desempeño)** Utilizar el modelo del inciso anterior para predecir la clase de cada una de las imágenes del conjunto de evaluación de la cátedra, y entregar esas predicciones (ver "Acerca de la entrega"). La cátedra tiene las clases verdaderas de estas imágenes y va a calcular la *accuracy* real de su modelo sobre ellas (en este inciso se usa la *accuracy*, independientemente de las métricas elegidas en el inciso b). Antes de conocer ese resultado, ustedes deben responder: ¿Qué *accuracy* esperan que tenga su modelo sobre el conjunto de evaluación de la cátedra? Reportar un único valor y justificar cómo lo obtuvieron: ¿Sobre qué datos lo calcularon? ¿Por qué creen que ese valor representa bien lo que va a pasar con imágenes que el modelo nunca vió? ¿Esperan que la *accuracy* real sea mayor, menor o parecida a la que obtuvieron durante la selección del modelo (inciso c)? ¿Por qué?
    Importante: En este inciso NO se evalúa quién obtiene el mejor modelo, sino qué tan cerca queda su estimación de la *accuracy* real. Un modelo con menor *accuracy* pero bien estimada es una mejor respuesta que un modelo con mayor *accuracy* pero mal estimada. Tengan especial cuidado de no usar, para estimar el desempeño, datos que el modelo ya haya visto durante el entrenamiento o la selección de hiperparámetros.

Importante (reproducibilidad): varios pasos de este TP dependen del azar, por ejemplo la separación en *train* y *test* o *dev* y *held-out*, la partición en *folds* de la validación cruzada, la elección de atributos al azar y algunos modelos. Para que sus resultados puedan replicarse, deben fijar una semilla al principio del código (por ejemplo, SEMILLA = 42) y utilizarla en todas las operaciones que involucren azar (por ejemplo, en el parámetro *random_state* de las funciones de *scikit-learn*). Al volver a correr el código completo se deben obtener exactamente los mismos resultados que figuran en el informe.

Grupos

Los grupos deben estar conformados por 3 (y sólo 3) integrantes. Ni más, ni menos. Pueden cambiar de grupo. Deberán i) registrar la conformación del grupo en la siguiente planilla, y ii) definir quién va a ser el encargado del envío (debe ser uno y sólo uno de los integrantes del grupo): LaboDatos 2026C2 TP-02-Grupos
NOTA: Si mantienen la conformación del grupo tal cual la del TP-01, por favor mantengan el MISMO nombre.

Acerca de la entrega

Para la entrega deberán preparar los siguientes archivos:

1. Un archivo llamado TP-02-nombregrupo.py con el código principal. Este archivo puede complementarse con otros archivos .py donde figure parte del código, y que sean importados y utilizados desde el archivo principal.
   Como siempre, ordenar el código de la siguiente manera:
   - Al inicio, un encabezado con una descripción que contemple: el nombre del grupo, los nombres de lxs participantes, contenido del archivo y cualquier otro dato relevante que considere importante.
   - Luego la sección de los imports.
   - A continuación, la definición de la semilla que se usará en todo el código (ver "Importante (reproducibilidad)" en Ejercicios).
   - Después, la carga de datos.
   - Siguiendo, las funciones propias que hayan definido.
   - Y finalmente, el código que no está dentro de funciones.

   El código debe estar modularizado (separando bloques con #%%) para permitir su ejecución por fragmentos. Todo lo que figure en el informe debe deducirse de los resultados del código, y al volver a ejecutarlo se deben obtener los mismos valores.
2. Un archivo llamado README.txt con los requerimientos de bibliotecas utilizadas e instrucciones de cómo ejecutar el código.
3. Un informe breve (no más de 10 carillas) en pdf llamado TP-02-Informe-nombregrupo.pdf. Además deben entregar una copia impresa (el día indicado en el Enunciado).
   Ordenar el informe de la siguiente manera:
   - Breve introducción al problema donde se muestre el análisis exploratorio realizado.
   - Explicación sobre los experimentos realizados, incluyendo los gráficos que consideren convenientes.
   - Conclusiones, incluyendo los resultados relevantes de los modelos desarrollados y la accuracy estimada en el inciso 3.e, junto con su justificación.
   - Bibliografía. Citen el paper [1] y cualquier otra fuente que hayan utilizado.
4. Un archivo llamado TP-02-<performance>-nombregrupo.csv con las predicciones del inciso 3.e sobre el conjunto de evaluación de la cátedra. Debe tener exactamente

dos columnas: id (el identificador de cada imagen, tal como figura en el archivo provisto) y label (la clase predicha, un número entre 0 y 9), con una fila por cada imagen del conjunto. En el nombre del archivo, <performance> tiene que ser reemplazado por la accuracy estimada en el inciso 3.e, redondeada a 4 decimales y usando punto como separador decimal. Por ejemplo, si el grupo "predictor trivial" estimó una accuracy de 0.67, el archivo debe llamarse TP-02-0.6700-predictor trivial.csv, es fundamental que respeten este formato para la evaluación.

5. La planilla de autoevaluación que se explica a continuación.

Importante: No deben entregar los archivos del dataset.

Autoevaluación

Al finalizar la entrega, y antes de enviar el TP-02, realizar lo siguiente:

1. Copiar la siguiente planilla de autoevaluación (una sola a nivel grupal) a una carpeta personal: TP-02-Autoevaluacion
2. Completarla.
3. Descargar como pdf y agregarla al envío virtual y en papel.

Referencias

[1] Ha, David, and Douglas Eck. "A Neural Representation of Sketch Drawings." arXiv preprint arXiv:1704.03477 (2017).
[2] Google Creative Lab, "The Quick, Draw! Dataset," *GitHub*, 2017. [En línea]. Disponible en: https://github.com/googlecreativelab/quickdraw-dataset